

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CMC**



HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU

Tên Đề tài : Phát hiện bất thường mạng trên CIC-IDS2018

Ngành : Công nghệ thông tin

Giáo viên hướng dẫn : Nguyễn Hữu Quỳnh, Bùi Tuấn Minh

Nhóm Sinh viên :

- | | |
|---------------------------------|----------------------------------|
| 1. Vũ Tuấn Đạt | Mã SV: BIT240055 - Lớp: 24IT-GM. |
| 2. Vũ Minh Đức | Mã SV: BIT240066 - Lớp: 24IT-GM |
| 3. Hoàng Phúc Vinh | Mã SV: BIT240255 – Lớp: 24IT-GM |
| 4. Nguyễn Ngọc Tuấn Linh | Mã SV: BIT240140 – Lớp: 24IT-GM |
| 5. Mai Đức Minh | Mã SV : BIT240151 - Lớp: 24IT-GM |

Hà Nội, tháng 5 năm 2026

MỤC LỤC

MỤC LỤC.....	1
LỜI MỞ ĐẦU.....	3
TUẦN 2 : XÂY DỰNG MÔ HÌNH MLP BASELINE.....	4
1. Bài toán :	4
2. Dataset :	5
3. Pipeline :	7
4. Model – Các kiến trúc đã thử nghiệm :	9
5. So sánh kết quả :	12
6. Failure Analysis :	13
TUẦN 3 : XÂY DỰNG MẠNG CNN PHÂN LOẠI NẤM.....	15
1. Bài toán :	15
2. Dataset :	16
3. Pipeline :	19
4. Model – Kiến trúc CNN đã thử nghiệm :	22
5. So sánh kết quả :	26
6. Failure Analysis :	28
TUẦN 6 : MÔ HÌNH RNN SINH VĂN BẢN THEO KÝ TỰ.....	31
1. Bài toán.....	31
2. Dataset.....	32
3. Pipeline.....	35
4. Model – Kiến trúc đã thử nghiệm.....	37
5. So sánh kết quả.....	39
6. Failure Analysis.....	41
TUẦN 7 : TRANSFORMER AUTOENCODER PHÁT HIỆN TẤN CÔNG MẠNG.....	43
1. Bài toán.....	43
2. Dataset.....	44
3. Pipeline.....	47
4. Model – Kiến trúc Transformer Autoencoder.....	49
5. So sánh kết quả.....	51
TUẦN FINAL : PHÁT HIỆN BẤT THƯỜNG MẠNG.....	52
2. Dataset.....	52
3. Pipeline.....	54
4. Model – Các kiến trúc đã thử nghiệm.....	57
5. So sánh kết quả.....	62
6. Failure Analysis.....	64
KẾT LUẬN.....	67

LỜI MỞ ĐẦU

Trong bối cảnh các hệ thống thông tin ngày càng mở rộng về quy mô và mức độ kết nối, những hình thức tấn công mạng cũng trở nên đa dạng, tinh vi và khó phát hiện hơn. Các phương pháp giám sát dựa trên luật cố định thường gặp hạn chế khi phải xử lý khối lượng lưu lượng lớn hoặc những hành vi bất thường chưa được định nghĩa trước. Vì vậy, ứng dụng học máy và học sâu vào phát hiện xâm nhập mạng là một hướng tiếp cận có nhiều tiềm năng.

Trong đồ án này, nhóm xây dựng một pipeline phát hiện bất thường mạng trên tập dữ liệu CSE-CIC-IDS2018. Bài toán được thiết lập dưới dạng phân loại nhị phân, trong đó mỗi network flow được xác định là lưu lượng bình thường (**BENIGN**) hoặc lưu lượng tấn công (**ATTACK**) dựa trên các đặc trưng thống kê về gói tin, thời gian, tốc độ truyền và trạng thái kết nối.

Quá trình thực hiện đồ án được phát triển qua nhiều giai đoạn, bắt đầu từ việc xây dựng mô hình MLP baseline, nghiên cứu CNN, Transfer Learning, RNN và Transformer. Trên cơ sở những kiến thức đã tích lũy, nhóm tiến hành thực nghiệm cuối cùng với bốn kiến trúc gồm MLP, CNN1D, BiLSTM và Transformer. Các mô hình được huấn luyện và đánh giá trên cùng một tập dữ liệu, quy trình tiền xử lý và cấu hình thực nghiệm nhằm bảo đảm tính nhất quán trong quá trình so sánh.

Báo cáo tập trung trình bày bài toán, đặc điểm dữ liệu, pipeline xử lý, kiến trúc mô hình, kết quả thực nghiệm và phân tích các trường hợp dự đoán sai. Bên cạnh hiệu năng phân loại, nhóm cũng xem xét số lượng tham số, thời gian huấn luyện, khả năng giải thích và mức độ phù hợp của từng kiến trúc đối với dữ liệu network flow dạng bảng. Qua đó, nhóm hướng đến việc lựa chọn một mô hình có sự cân bằng giữa độ chính xác, chi phí tính toán và khả năng triển khai thực tế.

Kết quả thực nghiệm cho thấy mô hình MLP đạt hiệu quả tổng thể tốt nhất với Accuracy và Macro-F1 đạt 96,79%, ROC-AUC đạt 98,59% và PR-AUC đạt 99,00%. Kết quả này cho thấy kiến trúc phù hợp với đặc điểm dữ liệu có thể quan trọng hơn việc chỉ gia tăng độ phức tạp của mô hình. Đồng thời, việc phân tích các trường hợp dự đoán sai cũng chỉ ra rằng hệ thống vẫn cần được cải thiện về khả năng phát hiện đầy đủ các lưu lượng tấn công.

Thông qua đề án, nhóm không chỉ xây dựng được một hệ thống phát hiện bất thường mạng chạy hoàn chỉnh từ khâu xử lý dữ liệu đến đánh giá và lưu mô hình, mà còn có cơ hội vận dụng, so sánh và đánh giá nhiều kiến trúc học sâu trong cùng một bài toán thực tế. Đây là nền tảng để nhóm tiếp tục phát triển hệ thống theo hướng phân loại đa lớp, xử lý dữ liệu theo chuỗi thời gian và triển khai trong môi trường mạng thực tế.

TUẦN 2 : XÂY DỰNG MÔ HÌNH MLP BASELINE

1. Bài toán :

1.1. Mục tiêu :

+ Mục tiêu của tuần 2 là xây dựng một mô hình học sâu baseline có khả năng dự đoán trạng thái việc làm của sinh viên sau khi hoàn thành chương trình học.

+ **Bài toán được phát biểu như sau:**

- Dựa trên các thông tin về quá trình học tập, mức độ chuyên cần và thói quen sinh hoạt của sinh viên, mô hình dự đoán sinh viên thuộc nhóm “**Placed**” hay “**Not Placed**”.

+ **Kết quả dự đoán có thể được sử dụng để:**

- Nhận diện sớm sinh viên có nguy cơ chưa tìm được việc làm.
- Hỗ trợ nhà trường xây dựng kế hoạch tư vấn và đào tạo bổ sung.
- Phân tích những yếu tố có liên quan đến khả năng tìm được việc làm.
- Tạo mô hình baseline để làm cơ sở so sánh với các kiến trúc phức tạp hơn.

1.2. Loại bài toán :

Đây là bài toán phân loại nhị phân có giám sát:

Trong đó:

- (X) là vector gồm 7 đặc trưng của sinh viên.
- (y) là nhãn đầu ra thuộc một trong hai lớp:
 1. **Not Placed:** sinh viên chưa có việc làm.
 2. **Placed:** sinh viên đã có việc làm.

2. Dataset :

2.1. Tổng quan dữ liệu :

- + **Dataset được lưu dưới dạng CSV với tên:**
student_dataset_10000_rows.csv
- + **Tập dữ liệu gồm :**
 - 10.000 mẫu sinh viên.
 - 7 đặc trưng đầu vào.
 - 1 biến mục tiêu.

2.2. Các thuộc tính :

Thuộc tính	Ý nghĩa	Vai trò
study_hours	Số giờ học của sinh viên	Đầu vào
attendance	Mức độ hoặc tỷ lệ tham gia lớp học	Đầu vào
sleep_hours	Số giờ ngủ	Đầu vào
internet_usage	Mức độ sử dụng Internet	Đầu vào
assignments_completed	Số lượng bài tập đã hoàn thành	Đầu vào
previous_score	Điểm số trước đó	Đầu vào
exam_score	Điểm thi hiện tại	Đầu vào
placement_status	Trạng thái việc làm	Nhãn dự đoán

- Các đặc trưng đầu vào đều được xử lý dưới dạng số liên tục trước khi đưa vào mạng MLP.

2.3. Phân bố nhãn :

Từ confusion matrix trên tập test, phân bố nhãn được xác định như sau:

Nhãn	Số mẫu test	Tỷ lệ
Not placed	329	16,45%
Placed	1.671	83,55%
Tổng	2.000	100%

+ Dữ liệu có hiện tượng mất cân bằng lớp. Lớp **Placed** nhiều hơn lớp **Not Placed** khoảng 5 lần.

+ Notebook sử dụng **stratify** khi phân chia dữ liệu, vì vậy tỷ lệ hai lớp được duy trì gần tương đương giữa train, validation và test.

+ Nếu phân bố toàn bộ dữ liệu tương ứng chính xác với phân bố test, dataset sẽ có xấp xỉ:

- 1.645 mẫu **Not Placed**.
- 8.355 mẫu **Placed**.

+ Hai con số trên là ước lượng từ tập test; cần đọc lại file CSV gốc để xác nhận tuyệt đối.

2.4. Phân chia dữ liệu :

+ Dữ liệu được chia thành ba tập độc lập:

Tập dữ liệu	Số mẫu	Tỷ lệ	Mục đích
Train	6.000	60%	Học trọng số mô hình
Validation	2.000	20%	Theo dõi khả năng khái quát trong quá trình huấn luyện
Test	2.000	20%	Đánh giá cuối cùng
Tổng	10.000	100%	—

+ Quá trình chia được thực hiện hai lần:

1. Chia 80% development và 20% test.
2. Chia development thành 75% train và 25% validation.

→ **Kết quả cuối cùng là tỷ lệ 60/20/20.**

2.5. Tiền xử lý dữ liệu :

+ Nhãn văn bản được chuyển thành số bằng **LabelEncoder**:

- **Not Placed** → 0.
- **Placed** → 1.

+ Các đặc trưng được chuẩn hóa bằng **StandardScaler**.

+ Scaler chỉ được **fit** trên tập train. Tập validation và test chỉ sử dụng phép **transform**. Cách triển khai này giúp hạn chế rò rỉ thông tin thống kê từ validation hoặc test vào quá trình huấn luyện.

3. Pipeline :

3.1. Quy trình tổng thể :

File CSV trên Google Drive

|



Đọc bằng Pandas

|



Tách đặc trưng X và nhãn y

|

|— X: 7 đặc trưng

|— y: placement_status

|



LabelEncoder cho nhãn

|



Chia dữ liệu có stratify

|

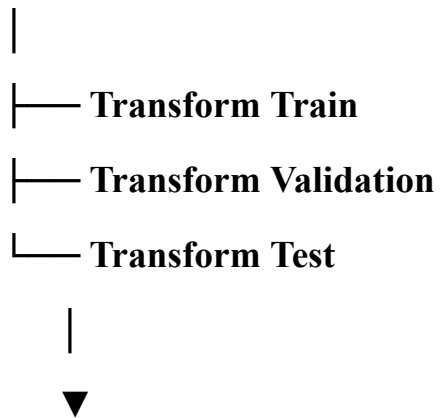


Train Val Test

60% 20% 20%



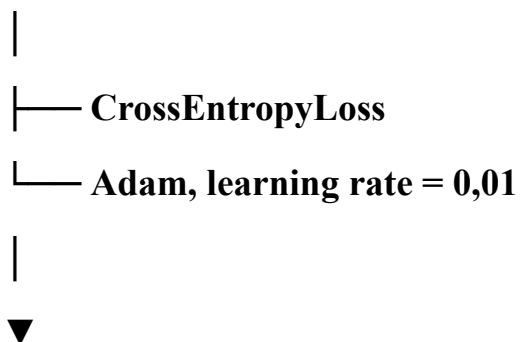
Fit StandardScaler trên Train



Chuyển dữ liệu thành Tensor



Huấn luyện MLP 90 epoch

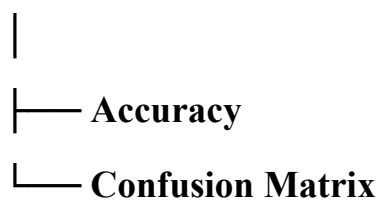


Theo dõi Train/Validation Loss

và Train/Validation Accuracy



Đánh giá một lần trên tập Test



3.2. Quy trình huấn luyện mỗi epoch :

Trong mỗi epoch, hệ thống thực hiện:

1. Chuyển mô hình sang chế độ **train**.
2. Forward toàn bộ tập train qua mạng MLP.
3. Tính Cross-Entropy Loss.
4. Xóa gradient cũ.
5. Lan truyền ngược.
6. Cập nhật trọng số bằng Adam.
7. Chuyển mô hình sang chế độ **eval**.
8. Dự đoán trên tập validation.
9. Lưu train loss, validation loss và accuracy.
10. Lặp lại quy trình trong 90 epoch.

+ Notebook đang sử dụng full-batch training, nghĩa là toàn bộ 6.000 mẫu train được đưa qua mô hình trong một lần cập nhật ở mỗi epoch.

4. Model – Các kiến trúc đã thử nghiệm :

4.1. Mô hình MLP baseline :

+ Mô hình chính của tuần 2 là mạng neuron truyền thẳng đa tầng:

Input: 7 đặc trưng

|



Linear($7 \rightarrow 32$)

|

LeakyReLU

|



Linear($32 \rightarrow 16$)

|

LeakyReLU

|



Linear($16 \rightarrow 2$)



Hai output logits

Cấu hình huấn luyện:

Thành phần	Giá trị
Input dimension	7
Hidden layer 1	32 neuron
Hidden layer 2	16 neuron
Output layer	2 neuron
Activation	LeakyReLU
Loss	CrossEntropyLoss
Optimizer	Adam
Learning rate	0,01
Số epoch	90
Batch strategy	Full batch

→ **MLP baseline có tổng cộng 818 tham số có thể huấn luyện.**

4.2. Thử nghiệm projection kết hợp head ResNet18 :

Notebook còn thực hiện một thử nghiệm bổ sung:

7 đặc trưng

|



Linear Projection

$7 \rightarrow 512$

|



ResNet18 FC Head

$512 \rightarrow 2$

|



Hai output logits

Quá trình thực hiện:

- Tải ResNet18 đã được huấn luyện trên ImageNet.
- Đóng băng tham số của backbone.
- Thay lớp fc cuối bằng **Linear(512, 2)**.
- Tạo lớp projection **Linear(7, 512)**.
- Huấn luyện projection và lớp fc trong 90 epoch.

+ Tuy nhiên, dữ liệu không đi qua các convolutional block của ResNet18. Mô hình chỉ sử dụng một lớp chiếu tuyến tính và lớp fully connected cuối.

→ Vì vậy, thử nghiệm này không nên được xem là transfer learning ResNet18 đúng nghĩa. Trọng số pretrained của backbone hầu như không tham gia vào dự đoán.

5. So sánh kết quả :

5.1. Kết quả huấn luyện :

Mô hình	Train loss đầu	Train loss cuối	Validation loss đầu	Validation loss cuối
MLP baseline	0,7242	0,0146	0,6392	0,0185
Projection + FC head	0,5110	0,0501	0,4331	0,0519

+ Loss của cả hai mô hình đều giảm ổn định. Đối với MLP, validation loss cuối chỉ cao hơn train loss khoảng:

$$+ 0,0185 - 0,0146 = 0,0039$$

→ Khoảng cách nhỏ cho thấy chưa có dấu hiệu overfitting nghiêm trọng trên cách chia dữ liệu hiện tại.

5.2. Hiệu năng trên tập test :

Mô hình	Test accuracy	Số dự đoán đúng	Số dự đoán sai
MLP baseline	99,15%	1.983/2.000	17/2.000
Projection + FC head	99,65%	1.993/2.000	7/2.000

+ Mô hình projection cao hơn MLP:

$$\rightarrow 99,65\% - 99,15\% = 0,50 \text{ (điểm phần trăm)}$$

+ Số lỗi giảm từ 17 xuống 7, tương đương giảm 10 lỗi trên tập gồm 2.000 mẫu.

+ Tuy nhiên, chưa nên kết luận thử nghiệm thứ hai tốt hơn một cách tổng quát vì:

- Chỉ có một lần chia dữ liệu.
- Chưa đánh giá qua nhiều random seed.
- Chưa có kiểm định thống kê.
- Hai mô hình sử dụng cấu hình tối ưu khác nhau.
- Kết quả thứ hai không thực sự đến từ backbone pretrained **ResNet18**.

5.3. Chỉ số chi tiết của MLP :

Confusion matrix:

Nhân thực tế \ Nhân dự đoán	Not Placed	Placed
Not Placed	317	12
Placed	5	1.666

→ Mặc dù lớp **Not Placed** ít hơn đáng kể, MLP vẫn nhận diện đúng 96,35% số mẫu thuộc lớp này.

6. Failure Analysis :

6.1. Phân tích confusion matrix

Trong 2.000 mẫu test, MLP dự đoán sai 17 mẫu:

- 12 sinh viên thực tế **Not Placed** nhưng được dự đoán là **Placed**.
- 5 sinh viên thực tế **Placed** nhưng được dự đoán là **Not Placed**.

Phân bố lỗi:

1. **Not placed** → **Placed** : 12 lỗi (70,59%).

2. **Placed** → **Not placed** : 5 lỗi (29,41%).

→ Mô hình có xu hướng dự đoán nhầm lớp thiếu số **Not Placed** sang lớp đa số **Placed**.

→ Nếu mục tiêu triển khai là phát hiện sinh viên cần hỗ trợ, 12 trường hợp này quan trọng hơn 5 lỗi theo chiều ngược lại. Đây là những sinh viên thực tế chưa có việc làm nhưng hệ thống đánh giá rằng họ đã có việc làm, dẫn đến nguy cơ không nhận được hỗ trợ cần thiết.

6.2. Diễn biến loss :

Train loss và validation loss cùng giảm trong 90 epoch:

- Train loss: 0,7242 → 0,0146.
- Validation loss: 0,6392 → 0,0185.
- Hai đường loss không phân kỳ rõ rệt.
- Validation loss tiếp tục giảm đến epoch cuối.

→ Kết quả cho thấy quá trình tối ưu hội tụ tốt trên tập dữ liệu hiện tại. Tuy nhiên, loss thấp bất thường và accuracy trên 99% cũng đặt ra câu hỏi về độ khó thực sự của dữ liệu.

6.3. Nguy cơ mất cân bằng lớp :

→ Lớp **Placed** chiếm 83,55% tập test.

→ Vì vậy, accuracy không nên là chỉ số duy nhất. Báo cáo cần bổ sung:

- Precision, recall và F1 của từng lớp.
- Macro F1.
- Balanced accuracy.
- ROC-AUC và PR-AUC.
- Đặc biệt theo dõi recall của lớp **Not Placed**.

6.4. Nguy cơ rò rỉ nhãn :

Đặc trưng **exam_score** cần được kiểm tra kỹ.

Nếu **placement_status** trong dataset được sinh ra dựa trực tiếp hoặc gần như trực tiếp từ **exam_score**, việc sử dụng điểm thi làm đầu vào sẽ khiến mô hình nhìn thấy thông tin gần tương đương với nhãn cần dự đoán.

Điều này có thể giải thích cho:

- Accuracy cao hơn 99%.
- Loss rất thấp.
- Mô hình chỉ có 818 tham số nhưng phân loại gần như hoàn hảo.
- Cả hai kiến trúc tương đối đơn giản đều đạt kết quả rất cao.

Hiện notebook chưa thực hiện:

- Phân tích tương quan giữa từng feature và nhãn.
- Kiểm tra quy tắc sinh nhãn.
- Ablation study khi loại **exam_score**.
- Phân tích trực tiếp 17 mẫu bị dự đoán sai.

Vì vậy, chưa thể kết luận accuracy 99,15% sẽ được duy trì trên dữ liệu sinh viên thực tế ngoài dataset.

6.5. Giới hạn của failure analysis hiện tại :

Notebook chỉ lưu confusion matrix, không lưu:

- Chỉ số hoặc dữ liệu của 17 mẫu sai.
- Xác suất dự đoán.
- Mức độ tự tin của từng dự đoán.
- Phân bố đặc trưng giữa mẫu đúng và mẫu sai.

Do đó, báo cáo có thể phân tích hướng sai và số lượng lỗi, nhưng chưa thể kết luận chính xác đặc trưng nào gây ra từng lỗi.

TUẦN 3 : XÂY DỰNG MẠNG CNN PHÂN LOẠI NẤM

1. Bài toán :

1.1. Mục tiêu :

→ Mục tiêu của tuần 3 là xây dựng một mạng neuron tích chập CNN có khả năng nhận ảnh đầu vào và phân loại ảnh đó vào một trong tám loài nấm có trong tập dữ liệu.

1.1.2. Bài toán được phát biểu như sau:

- Cho một ảnh RGB chứa nấm, hệ thống cần học các đặc trưng hình ảnh như màu sắc, hình dạng, kết cấu và cấu trúc của cây nấm để dự đoán loài tương ứng.

- Trong đó:

- (X) là một ảnh RGB.
- (y) là nhãn loài nấm.
- Mỗi ảnh chỉ thuộc một trong tám lớp.

1.2. Đầu vào và đầu ra :

Ứng dụng tiềm năng của hệ thống:

- Hỗ trợ nhận diện sơ bộ các loài nấm từ ảnh.
- Tự động phân loại ảnh trong kho dữ liệu.
- Hỗ trợ nghiên cứu đặc điểm hình thái của nấm.
- Tạo baseline CNN để so sánh với transfer learning trong tuần tiếp theo.

→ Mô hình chỉ thực hiện phân loại hình ảnh và không đủ cơ sở để kết luận một loại nấm có ăn được hoặc có độc hay không.

2. Dataset :

2.1. Nguồn dữ liệu :

+ **Notebook tải dataset từ Kaggle:**

parvezkabir221155539/mushroom-species-classification-dataset.

+ Dataset gốc có dung lượng nén khoảng **4,97 GB**. Tuy nhiên, sau khi duyệt các thư mục ảnh, notebook chỉ ghi nhận **743** ảnh hợp lệ thuộc tám lớp.

2.2. Cấu trúc dữ liệu :

+ Dữ liệu được tổ chức theo thư mục:

Mashroom Dataset/

├── Ear Mushroom/

├── Golden Oyster Mushroom/

├── Gray Oyster Mushroom/

├── Lions Mane Mushroom/

├── MilkyMushroom/

├── Oyster (Po10) Mushroom/

├── Pink Oyster Mashroom/

└── Reishi Mushroom/

+ Tên thư mục được sử dụng làm nhãn của ảnh. Notebook tạo một DataFrame gồm hai cột:

Thuộc tính	Mô tả
image_path	Đường dẫn đến file ảnh
label	Tên lớp lấy từ thư mục chứa ảnh
label_encoded	Nhãn số từ 0 đến 7 được tạo sau đó

→ Kiểm tra ban đầu cho thấy:

- Tổng số ảnh: 743.
- Tổng số lớp: 8.
- Số giá trị thiếu: 0.
- Số hàng trùng lặp theo **image_path** và **label**: 0.

+ Lưu ý rằng kiểm tra **DataFrame.duplicated()** chỉ phát hiện hàng có cùng đường dẫn và nhãn. Phép kiểm tra này chưa phát hiện được các ảnh trùng nội dung nhưng có tên file khác nhau.

2.3. Ánh xạ nhãn :

Mã lớp	Tên lớp
0	Ear Mushroom
1	Golden Oyster Mushroom
2	Gray Oyster Mushroom
3	Lions Mane Mushroom
4	MilkyMushroom
5	Oyster (Po10) Mushroom
6	Pink Oyster Mashroom
7	Reishi Mushroom

+ Một số tên lớp trong dataset có lỗi chính tả hoặc khoảng trắng không thống nhất, chẳng hạn **Mashroom**, **MilkyMushroom** và hai khoảng trắng trong tên Golden Oyster. Điều này không làm hỏng quá trình huấn luyện hiện tại nhưng cần chuẩn hóa để báo cáo và triển khai nhất quán.

2.4. Phân bố dữ liệu :

+ Các số lượng được notebook hiển thị rõ:

Loài nấm	Số ảnh	Tỉ lệ xấp xỉ
Oyster (Po10) Mushroom	230	30,96%
Pink Oyster Mashroom	225	30,28%
Reishi Mushroom	73	9,83%
Ear Mushroom	72	9,69%
Golden Oyster Mushroom	57	7,67%
Các lớp còn lại	86	11,57%
Tổng	743	100%

+ Ba lớp còn lại là:

- Gray Oyster Mushroom.
- MilkyMushroom.
- Lions Mane Mushroom.

+ Dựa trên tỷ lệ phân chia được lưu trong notebook, số lượng toàn bộ dataset của ba lớp này xấp xỉ:

Loài nấm	Số ảnh ước tính
Gray Oyster Mushroom	44
MilkyMushroom	26
Lions Mane Mushroom	16

→ Điều này khiến mô hình có nguy cơ ưu tiên hai lớp lớn là Oyster và Pink Oyster, đồng thời học chưa đủ đặc trưng của **Lions Mane** và **MilkyMushroom**.

2.5. Phân chia dữ liệu :

Dataset được chia có **stratify** để duy trì tỷ lệ lớp:

Tập dữ liệu	Số ảnh	Tỷ lệ	Mục đích
Train	445	59,89%	Huấn luyện mô hình
Validation	149	20,05%	Theo dõi khả năng khái quát
Test	149	20,05%	Đánh giá cuối cùng
Tổng	743	100%	

+ Quy trình chia:

1. Tách 20% dữ liệu làm test.
2. Chia 80% còn lại thành 75% train và 25% validation.
3. Kết quả cuối gần bằng 60/20/20.

→ Do một số lớp có rất ít ảnh, tập test chỉ có khoảng ba ảnh Lions Mane và năm ảnh **MilkyMushroom**. Chỉ một dự đoán sai cũng làm recall của những lớp này thay đổi rất mạnh.

3. Pipeline :

3.1. Quy trình tổng thể :

Kaggle Mushroom Dataset

|



Tải file ZIP và giải nén

|



Duyệt thư mục của 8 lớp

|



Tạo DataFrame

(image_path, label)



Kiểm tra dữ liệu

└─ **Phân bố lớp**

└─ **Giá trị thiếu**

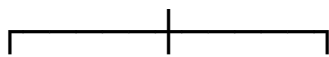
└─ **Hàng trùng lặp**



LabelEncoder: tên lớp → số 0–7



Chia dữ liệu có stratify



Train Val Test

60% 20% 20%



Tiền xử lý ảnh 224×224



└─ **Train: augmentation + normalize**

└─ Val/Test: resize + normalize

|



Tạo Dataset và DataLoader

|



Huấn luyện Custom CNN

└─ CrossEntropyLoss

└─ Adam

└─ 10 epoch dự kiến

|



Đánh giá trên tập test

|

└─ Accuracy

└─ Confusion matrix cần bổ sung

└─ Predict visualization cần bổ sung

3.2. Tiền xử lý và tăng cường dữ liệu

+ Đối với tập train:

Resize(224×224)

→ RandomHorizontalFlip

→ RandomRotation(10°)

→ ToTensor

→ Normalize theo ImageNet

+ Đối với validation và test:

Resize(224×224)

→ ToTensor

→ Normalize theo **ImageNet**.

+ Tăng cường dữ liệu chỉ áp dụng cho tập train. Đây là cách làm đúng vì validation và test cần phản ánh dữ liệu cố định để đánh giá khách quan.

+ Tuy nhiên, normalization theo ImageNet thường được sử dụng khi dùng pretrained model. Với CNN tự xây dựng từ đầu, nó vẫn có thể dùng được nhưng không phải lựa chọn bắt buộc. Tính mean và standard deviation trực tiếp từ tập train sẽ phù hợp với phân bố dữ liệu hơn.

3.3. DataLoader :

Tham số	Train	Validation	Test
Batch size	32	32	32
Shuffle	Có	Không	Không
Số batch	14	5	5

+ Mỗi batch ảnh có kích thước:

→ [32,3,224,224]

+ Train loader được trộn ngẫu nhiên, trong khi validation và test giữ nguyên thứ tự.

4. Model – Kiến trúc CNN đã thử nghiệm :

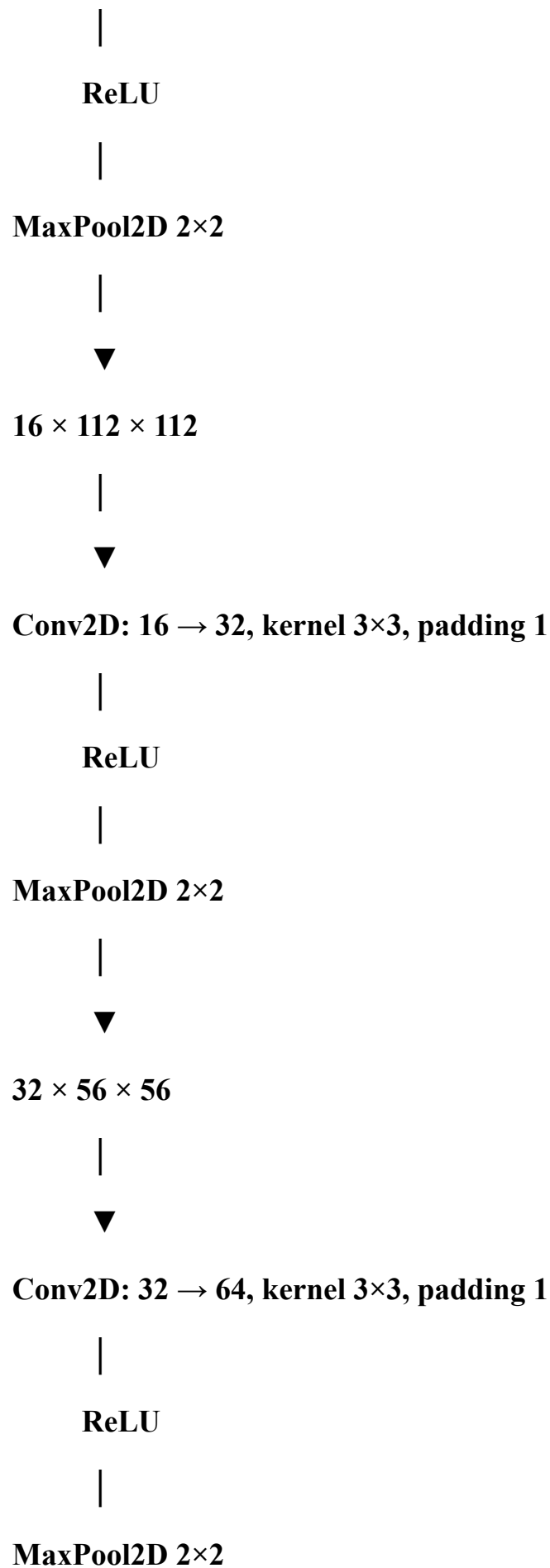
4.1. Kiến trúc tổng thể :

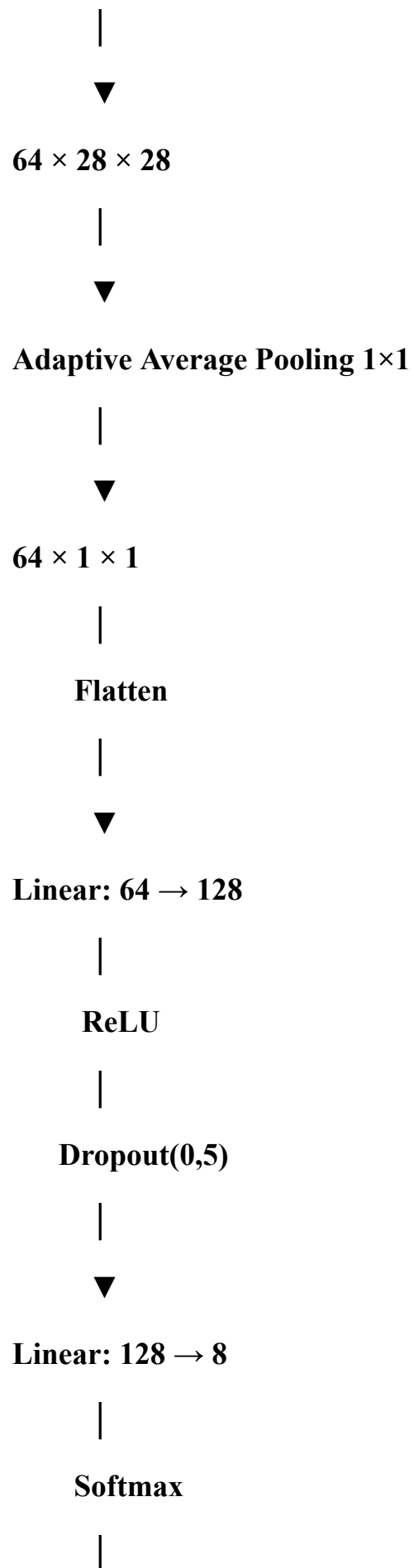
Input: $3 \times 224 \times 224$

|



Conv2D: $3 \rightarrow 16$, kernel 3×3 , padding 1







Dự đoán một trong 8 lớp

4.2. Số tham số :

Thành phần	Số tham số
Conv2D $3 \rightarrow 16$	448
Conv2D $16 \rightarrow 32$	4.640
Conv2D $32 \rightarrow 64$	18.496
Linear $64 \rightarrow 128$	8.320
Linear $128 \rightarrow 8$	1.032
Tổng	32.936

+ Mô hình có khoảng 32.936 tham số có thể huấn luyện. Đây là kiến trúc tương đối nhỏ, phù hợp làm CNN baseline.

4.3. Cấu hình huấn luyện :

Thành phần	Giá trị
Loss function	CrossEntropyLoss
Optimizer	Adam
Learning rate	0,001
Batch size	32
Epoch dự kiến	10
Dropout	0.5
Thiết bị trong lần chạy	GPU CUDA
Trọng số pretrained	Không

4.4. Vấn đề kỹ thuật trong output layer :

+ Mô hình hiện kết thúc bằng: **nn.Linear(128, num_classes),**

nn.Softmax(dim=-1).

+ Sau đó output được truyền vào: **nn.CrossEntropyLoss().**

→ Đây là cấu hình chưa chính xác. **CrossEntropyLoss** cần nhận trực tiếp logits thô vì bên trong nó đã thực hiện **LogSoftmax**.

→ Pipeline đúng nên là:

+ **Linear(128 → 8)**

→ logits

→ **CrossEntropyLoss(logits, label)**

+ Không nên thêm **Softmax** trong mô hình khi huấn luyện bằng **CrossEntropyLoss**.

+ Việc áp dụng **Softmax** hai lần về mặt logic làm gradient kém hiệu quả và có thể là một nguyên nhân khiến loss gần như không giảm trong kết quả hiện tại.

5. So sánh kết quả :

5.1. Trạng thái chạy thực tế :

→ Notebook được cấu hình chạy 10 epoch nhưng output chỉ hoàn thành hai epoch và dừng giữa epoch 3.

→ Do đó:

- Chưa có kết quả huấn luyện hoàn chỉnh.
- Chưa có test accuracy cuối cùng đáng tin cậy.
- Chưa có confusion matrix.
- Chưa có precision, recall và F1-score.
- Chưa có trực quan hóa dự đoán trên tập test.

5.2. Kết quả đã ghi nhận :

Epoch	Train loss	Train accuracy	Validation loss	Validation accuracy
1	2,0788	12,58%	2,0790	2,01%
2	2,0786	13,26%	2,0790	2,01%
3	Chưa hoàn thành	—	—	—

→ Loss quan sát được khoảng 2,079 cho thấy mô hình gần như đang đưa ra phân bố xác suất đồng đều cho tám lớp và chưa học được quy luật phân loại hữu ích.

5.3. So sánh với các baseline đơn giản :

Phương pháp	Accuracy
Dự đoán ngẫu nhiên cân bằng 8 lớp	12,50% kỳ vọng
Luôn dự đoán lớp lớn nhất	30,96%
CNN – Train epoch 1	12,58%
CNN – Train epoch 2	13,26%
CNN – Validation epoch 1–2	2,01%
CNN – Test	Chưa có

+ Train accuracy hiện chỉ xấp xỉ random baseline và thấp hơn nhiều so với majority-class baseline.

+ Đây cũng gần bằng số mẫu của lớp nhỏ nhất, **Lions Mane**. Mô hình có khả năng đang dự đoán phần lớn hoặc toàn bộ ảnh validation thành lớp Lions Mane. Cần kiểm tra trực tiếp vector dự đoán để xác nhận.

5.4. Bảng kết quả cần có sau khi chạy lại :

Mô hình	Train loss	Val loss	Accuracy	Macro F1	Weighted F1
CNN baseline hiện tại	Chưa hoàn chỉnh	Chưa hoàn chỉnh	Chưa có	Chưa có	Chưa có
CNN sửa bỏ Softmax	Cần chạy lại	Cần chạy lại	Cần chạy lại	Cần chạy lại	Cần chạy lại
Majority baseline	—	—	30,96%	Cần tính	Cần tính

→ Không nên đưa ra hoặc suy diễn test accuracy khi cell huấn luyện chưa chạy đến cuối.

6. Failure Analysis :

6.1. Loss không giảm :

+ Train loss : **2,0788** → **2,0786**

+ Validation loss : **2,0780** → **2,0790**

- Biểu đồ rút gọn:

Loss

2,080 ┆

2,079 ┆ Train ●——●

┆ Val ●——●

└──────────────────┘

1 2 Epoch

+ Loss hầu như không thay đổi. Kết luận trực tiếp là mô hình chưa học được sự phân tách giữa tám lớp trong hai epoch đầu.

+ Nguyên nhân có khả năng cao nhất là Softmax được đặt trước **CrossEntropyLoss**. Những nguyên nhân bổ sung gồm:

- Dataset nhỏ và mất cân bằng.
- Một số lớp chỉ có rất ít ảnh.
- Kiến trúc CNN tương đối nông.
- Adaptive average pooling có thể làm mất nhiều thông tin không gian.
- Chất lượng hoặc nội dung ảnh giữa các lớp có thể không đồng nhất.
- Chưa có Batch Normalization.
- Chưa kiểm tra learning rate và gradient.
- Chưa kiểm tra trực quan ảnh sau augmentation và normalization.

6.2. Mô hình không vượt qua baseline đơn giản

→ Accuracy train chỉ đạt 13,26%, trong khi dự đoán toàn bộ ảnh thành lớp lớn nhất đã có thể đạt khoảng 30,96%.

→ Điều đó có nghĩa kết quả hiện tại chưa đủ để xem mô hình là một baseline vận hành hiệu quả.

6.3. Ảnh hưởng của mất cân bằng lớp

+ Phân bố lớp rất chênh lệch:

Oyster (Po10)		230
Pink Oyster		225
Reishi		73
Ear Mushroom		72
Golden Oyster		57
Gray Oyster		44
MilkyMushroom		26
Lions Mane		16

→ Hai lớp lớn chiếm hơn 61% toàn bộ dataset. Trong khi đó, Lions Mane chỉ có khoảng 16 ảnh và khoảng chín ảnh được dùng để train.

→ Với số lượng nhỏ như vậy, mô hình khó học được biểu diễn ổn định cho lớp thiểu số. Accuracy tổng thể cũng có thể che giấu việc mô hình không nhận diện được các lớp nhỏ.

6.4. Confusion matrix chưa được triển khai :

+ Yêu cầu tuần 3 là lập confusion matrix, nhưng notebook hiện chỉ có hàm tính accuracy. Không có code:

- Thu thập toàn bộ nhãn thật.
- Thu thập dự đoán.
- Tính **confusion_matrix**.
- Vẽ heatmap tám lớp.

+ Do đó :

- Hai lớp nào dễ bị nhầm nhất.
- Mô hình có thiên lệch về một lớp hay không.
- Recall của các lớp ít dữ liệu.
- Mối quan hệ nhầm lẫn giữa các loài Oyster.

→ Dựa trên tên lớp, có thể đặt giả thuyết rằng các lớp Oyster dễ bị nhầm với nhau do cùng nhóm hình thái. Tuy nhiên, đây chỉ là giả thuyết và chưa được output hiện tại chứng minh.

6.5. Predict visualization chưa được triển khai

+ Notebook chưa hiển thị:

- Ảnh test.
- Nhãn thật.
- Nhãn dự đoán.
- Độ tin cậy.
- Đánh dấu dự đoán đúng hoặc sai.

+ Vì vậy, chưa thể phân tích các nguyên nhân trực quan như:

- Ảnh thiếu sáng hoặc bị mờ.
- Nấm chiếm diện tích quá nhỏ.
- Nền ảnh phức tạp.
- Góc chụp khác biệt.
- Ảnh chứa nhiều cây nấm.
- Màu sắc giữa các lớp tương tự.
- Nhãn dữ liệu có khả năng bị sai.

6.6. Nguy cơ trùng lặp và data leakage theo nội dung ảnh

+ Notebook chia dữ liệu ngẫu nhiên theo từng đường dẫn ảnh. Nếu dataset có:

- Nhiều ảnh gần giống nhau.
- Nhiều crop từ cùng ảnh gốc.
- Các frame liên tiếp của cùng video.
- Cùng cây nấm được chụp từ nhiều góc gần giống nhau.

→ Kiểm tra hàng trùng trong **DataFrame** không đủ để phát hiện vấn đề này. Cần dùng image hash, perceptual hash hoặc group split theo nguồn ảnh.

TUẦN 6 : MÔ HÌNH RNN SINH VĂN BẢN THEO KÝ TỰ

1. Bài toán

1.1. Mục tiêu

+ Mục tiêu của tuần 6 là xây dựng một mô hình xử lý dữ liệu chuỗi bằng mạng neuron hồi quy RNN. Hệ thống học cấu trúc văn bản Shakespeare và dự đoán ký tự tiếp theo dựa trên chuỗi ký tự đứng trước.

+ Bài toán được phát biểu như sau:

→ Cho một chuỗi gồm 50 ký tự, mô hình dự đoán ký tự tiếp theo tại mỗi vị trí, từ đó có thể sinh thêm văn bản theo phong cách của dữ liệu huấn luyện.

+ Đây là bài toán mô hình hóa ngôn ngữ ở cấp độ ký tự:

→ $P(x_{t+1} | x_1, x_2, \dots, x_t)$

+ Trong đó :

- x_t là ký tự tại thời điểm (t).
- Mô hình sử dụng trạng thái ẩn để tổng hợp thông tin từ các ký tự trước.
- Đầu ra là phân bố xác suất trên toàn bộ bộ từ vựng gồm 65 ký tự.

1.2. Đầu vào và đầu ra

+ Mỗi mẫu được tạo từ một đoạn gồm 51 ký tự:

- Đoạn gốc: "ROMEO:"
→ Input: "ROMEO"
→ Target: "OMEO:"

1.3. Ứng dụng

+ Mô hình minh họa các ứng dụng:

- Sinh văn bản tự động.
- Học cấu trúc chuỗi.
- Dự đoán ký tự tiếp theo.
- Phân tích sự thay đổi của hidden state theo thời gian.
- Làm baseline cho LSTM, GRU và Transformer.

→ Đây là mô hình học cấu trúc thống kê của văn bản, không thực sự hiểu nội dung hoặc ý nghĩa như con người.

2. Dataset

2.1. Nguồn dữ liệu

+ Notebook sử dụng tập văn bản Shakespeare được cung cấp qua TensorFlow:

<https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt>

+ Dữ liệu được tải bằng `tf.keras.utils.get_file`.

2.2. Quy mô dữ liệu

Thuộc tính	Giá trị
Tổng số ký tự	1.115.394
Số ký tự duy nhất	65
Độ dài input sequence	50
Độ dài đoạn trước khi tách	51
Batch size	64
Tổng số batch	341

+ Bộ từ vựng gồm 65 ký tự, bao gồm:

- Chữ cái viết hoa.

- Chữ cái viết thường.
- Chữ số hoặc ký hiệu xuất hiện trong văn bản.
- Dấu câu.
- Khoảng trắng.
- Ký tự xuống dòng.

2.3. Mã hóa ký tự

+ Các ký tự được sắp xếp và ánh xạ thành số nguyên:

1. **char2idx** : $c \rightarrow i$
2. **Idx2char** : $i \rightarrow c$

+ Ví dụ về quy trình:

- Ký tự $\rightarrow$ Chỉ số nguyên
- "R", "O", "M" $\rightarrow$ [iR, iO, iM]

2.4. Tạo chuỗi

+ Với batch size 64 và **drop_remainder=True**, notebook tạo được 341 batch:

$\rightarrow 341 \times 64 = 21.824$ sequence

+ Mỗi batch chứa: $64 \times 50 = 3200$ vị trí dự đoán ký tự.

2.5. Phân chia dữ liệu

Tập	Số batch	Tỷ lệ xấp xỉ	Số sequence
Train	204	59,82%	13.056
Validation	68	19,94%	4.352
Test	69	20,23%	4.416
Tổng	341	100%	21.824

+ Tổng số vị trí ký tự được đánh giá:

Tập	Số trị ví target
Train	652.800
Validation	217.600
Test	220.800

2.6. Vấn đề trong cách chia hiện tại

+ Notebook thực hiện **shuffle** trước khi dùng **take()** và **skip()** để tạo train, validation và test:

+ Dataset chuỗi

→ shuffle

→ batch

→ take/skip

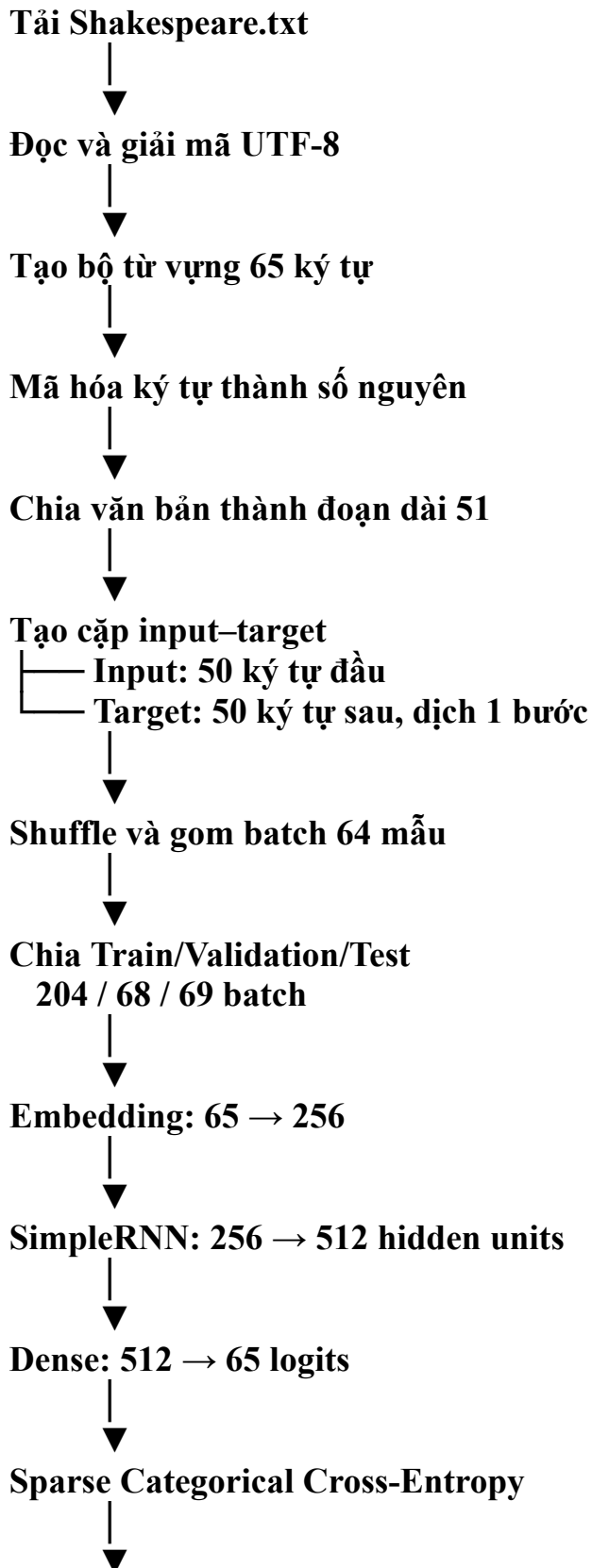
+ **tf.data.Dataset.shuffle()** mặc định có thể xáo lại dữ liệu ở mỗi lần duyệt. Vì train, validation và test được tạo bằng những pipeline take/skip riêng, tập dữ liệu thực tế có nguy cơ:

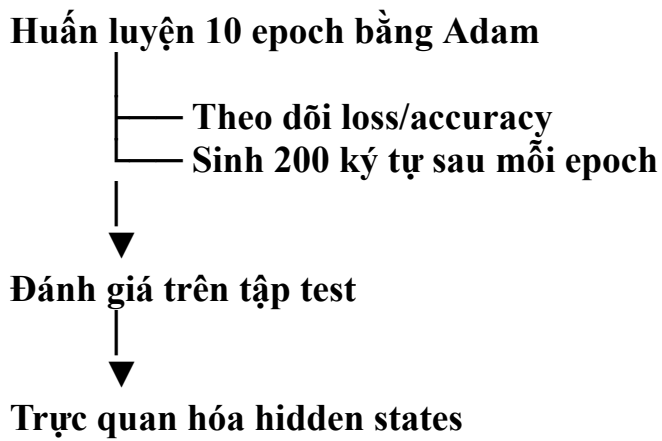
- Thay đổi giữa các epoch.
- Không duy trì ranh giới cố định.
- Chồng lấn sequence giữa train và validation/test.
- Làm kết quả đánh giá thiếu tin cậy.

→ Ngoài ra, với dữ liệu văn bản, chia ngẫu nhiên các đoạn từ cùng một tài liệu khiến train và test có phân bố gần như giống nhau. Một cách đánh giá nghiêm ngặt hơn là chia văn bản liên tục theo thời gian trước, sau đó mới tạo sequence riêng cho từng tập.

3. Pipeline

3.1. Quy trình tổng thể



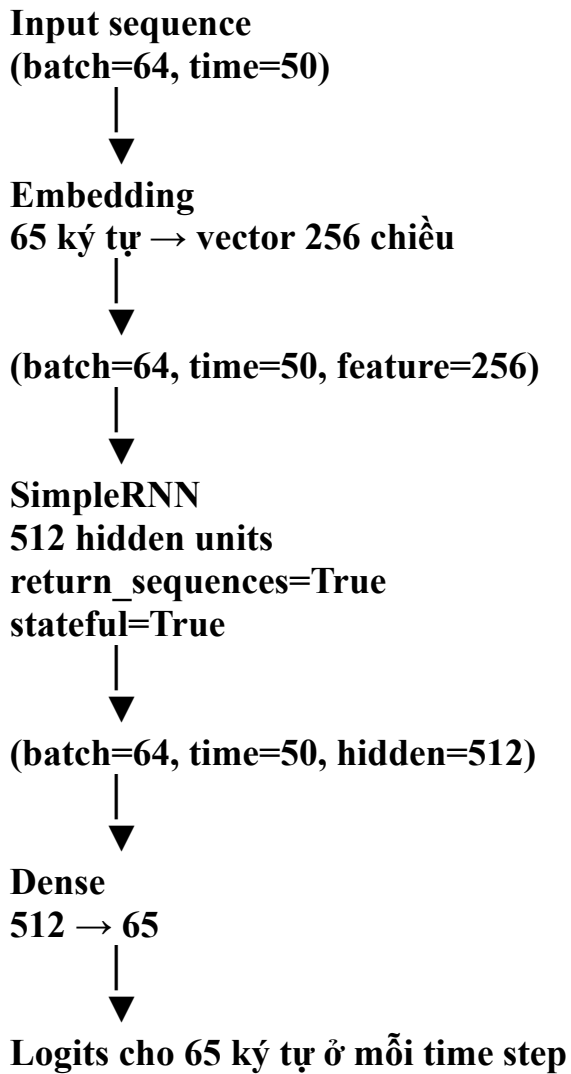


3.2. Pipeline sinh văn bản



4. Model – Kiến trúc đã thử nghiệm

4.1. Kiến trúc SimpleRNN



4.2. Thông số mô hình

Thành phần	Cấu hình
Vocabulary size	65
Embedding dimension	256
RNN type	SimpleRNN
Hidden units	512

Return sequences	Có
Stateful	Có
Output dimension	65
Batch size	64
Optimizer	Adam
Epoch	10

4.4. Số tham số

+ Embedding: $65 \times 256 = 16.640$

+ SimpleRNN: $(256 + 512 + 1) \times 512 = 393.728$

+ Dense: $512 \times 65 + 65 = 33.345$

+ Tổng: $16.640 + 393.728 + 33.345 = 443.713$

Layer	Số tham số
Embedding	16.640
SimpleRNN	393.728
Dense	33.345
Tổng	443.713

→ Toàn bộ tham số đều có thể huấn luyện.

5. So sánh kết quả

5.1. Kết quả theo epoch

Epoch	Train loss	Train accuracy	Validation loss	Validation accuracy
1	2,4069	33,34%	2,0704	39,76%
2	1,9681	42,89%	1,8886	44,71%
3	1,8109	47,00%	1,7643	47,99%
4	1,7178	49,39%	1,6949	49,81%
5	1,6535	51,14%	1,6366	51,48%
6	1,6113	52,21%	1,6079	52,19%
7	1,5760	53,12%	1,5669	53,16%
8	1,5519	53,74%	1,5511	53,66%
9	1,5277	54,38%	1,5274	54,26%
10	1,5131	54,70%	1,5214	54,54%

+ Kết quả trên tập test:

Chỉ số	Kết quả
Test loss	1,5115
Test accuracy	54,70%

5.2. Diễn biến loss

+ Train loss giảm: $2,4069 \rightarrow 1,5131$

+ Mức giảm tuyệt đối: $2,4069 - 1,5131 \rightarrow 0,8938$

+ Validation loss giảm: $2,0704 \rightarrow 1,5214$

→ Hai đường loss nằm gần nhau vào cuối quá trình huấn luyện. Điều này cho thấy chưa có dấu hiệu overfitting rõ ràng theo kết quả được lưu.

5.3. Diễn biến accuracy

+ Train accuracy tăng: 33,34% → 54,70%

+ Validation accuracy tăng: 39,76% → 54,54%

+ Khoảng cách cuối cùng: $54,70\% - 54,54\% = 0,16\%$

→ Test accuracy bằng 54,70%, gần với train và validation. Tuy nhiên, mức độ gần nhau bất thường này cũng cần được xem xét cùng nguy cơ chồng lấn do cách chia dataset.

5.4. So sánh chất lượng văn bản sinh ra

Giai đoạn	Đặc điểm
Epoch 1	Phần lớn là ký tự ngẫu nhiên, chưa tạo được từ
Epoch 2-3	Bắt đầu xuất hiện từ gần giống tiếng Anh
Epoch 4-6	Có cấu trúc câu và tên nhân vật nhưng nhiều từ sai
Epoch 7-8	Bắt đầu xuất hiện hội thoại và dấu câu hợp lý
Epoch 9-10	Văn bản có cấu trúc giống kịch Shakespeare nhưng chưa có nghĩa ổn định

+ Ví dụ cuối quá trình huấn luyện có dạng:

→ ROMEO: I'll my perform,

→ And bail and speak with ask against well...

+ **Mô hình đã học được:**

- Cách viết tên nhân vật.
- Dấu hai chấm sau tên.
- Cách xuống dòng.
- Một số mẫu ngữ pháp.

- Phân bố chữ cái và dấu câu.

+ Mô hình vẫn gặp các vấn đề:

- Tạo nhiều từ không tồn tại.
- Nội dung thiếu mạch lạc.
- Không duy trì được ngữ nghĩa dài hạn.
- Có thể đột ngột chuyển nhân vật hoặc chủ đề.

6. Failure Analysis

6.1. Phân tích lỗi dự đoán ký tự

→ Test accuracy 54,70% có nghĩa khoảng 45,30% vị trí ký tự bị dự đoán sai nếu chỉ chọn ký tự có xác suất lớn nhất.

+ Với 220.800 vị trí test, số lượng ước tính:

→ $220.80 \times 0,547 \sim 120.778$ (ký tự được dự đoán đúng)

→ $220.800 - 120.778 \sim 100.022$ (ký tự bị dự đoán sai).

6.2. Những loại lỗi quan sát được

+ Qua văn bản sinh sau mỗi epoch, có thể nhận diện các nhóm lỗi:

1. Ký tự ngẫu nhiên ở giai đoạn đầu :

+ Epoch đầu sinh nhiều chuỗi như: GQ3zYOLSSXVQXVONR...

→ Nguyên nhân là trọng số chưa học đủ quy luật chuyển tiếp giữa các ký tự.

2. Từ gần đúng nhưng không tồn tại :

+ Các từ như:

- fardar
- sentess
- carrind
- despive

→ Cho thấy mô hình đã học cấu trúc chính tả cục bộ nhưng chưa ghi nhớ được từ hoàn chỉnh.

3. Câu đúng hình thức nhưng sai ngữ nghĩa :

+ Mô hình tạo được:

- Tên nhân vật.
- Dấu câu.
- Xuống dòng.
- Một số cấu trúc hội thoại.

→ Tuy nhiên, các câu không duy trì ý nghĩa lâu dài. Đây là hạn chế thường gặp của SimpleRNN do khó lưu giữ quan hệ xa trong chuỗi.

6.3. Xung đột giữa `stateful=True` và shuffle

+ Mô hình dùng: **`stateful=True`**, nhưng dataset lại được shuffle.

→ Với `stateful` RNN, trạng thái cuối của một batch được truyền sang batch kế tiếp. Điều này chỉ hợp lý khi sample ở cùng vị trí trong batch kế tiếp là phần tiếp nối của sample ở batch trước.

+ Sau khi shuffle, quan hệ đó không còn được đảm bảo:

→ Batch A: đoạn từ Romeo

| trạng thái được giữ lại



→ Batch B: đoạn ngẫu nhiên từ Macbeth

+ Trạng thái từ một đoạn không liên quan bị đưa sang đoạn khác, tạo nhiễu cho quá trình học.

+ Hai cách sửa phù hợp:

- Dùng **`stateful=False`** với các sequence độc lập đã shuffle.
- Hoặc giữ **`stateful=True`**, không shuffle và tổ chức batch liên tục theo thời gian.

6.4. Nguy cơ chồng lấn train/validation/test

+ Do shuffle được đặt trước **`take/skip`**, các tập có thể không cố định giữa các lần lặp. Điều này có thể khiến một sequence xuất hiện trong train ở một lần duyệt và trong validation hoặc test ở lần khác.

→ Hệ quả:

- Validation loss có thể quá gần train loss.
- Test accuracy có thể bị đánh giá lạc quan.
- Không thể tái lập chính xác kết quả.
- So sánh mô hình thiếu công bằng.

TUẦN 7 : TRANSFORMER AUTOENCODER PHÁT HIỆN TẤN CÔNG MẠNG

1. Bài toán

1.1. Mục tiêu

→ Mục tiêu của tuần 7 là xây dựng mô hình Transformer để phát hiện lưu lượng mạng bất thường, cụ thể là các cuộc tấn công DDoS, dựa trên các đặc trưng của network flow.

+ Bài toán được phát biểu như sau :

→ Cho một bản ghi lưu lượng mạng, hệ thống xác định bản ghi đó thuộc lớp **BENIGN** (bình thường) hay **ATTACK** (tấn công).

+ Notebook tiếp cận bài toán theo hướng phát hiện bất thường bán giám sát:

1. Chỉ sử dụng các mẫu **BENIGN** trong tập train để huấn luyện Transformer Autoencoder.
2. Mô hình học cách tái tạo lưu lượng bình thường.
3. Dùng reconstruction error làm anomaly score.
4. Mẫu có anomaly score vượt ngưỡng được phân loại là **ATTACK**.

1.2. Loại bài toán

+ Đây là bài toán phân loại nhị phân:

→ $f(X) \rightarrow y$; $y \in \{0,1\}$

+ Trong đó :

- ($y=0$): lưu lượng **BENIGN**.
- ($y=1$): lưu lượng **ATTACK**.

1.3. Ý nghĩa ứng dụng

+ Hệ thống có thể hỗ trợ:

- Phát hiện sớm lưu lượng mạng bất thường.
- Cảnh báo các cuộc tấn công DDoS.
- Giảm khối lượng lưu lượng cần chuyên gia kiểm tra.
- Phát hiện mẫu bất thường mà không cần dùng nhãn tấn công trong quá trình huấn luyện autoencoder.
- Làm nền tảng cho hệ thống Network Intrusion Detection System.

→ Mô hình hiện chỉ phân biệt **BENIGN** và **ATTACK**, chưa xác định chi tiết loại hoặc nguồn gốc cuộc tấn công.

2. Dataset

2.1. Nguồn dữ liệu

+ Notebook sử dụng file:

Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv

+ File được tải từ bản sao dataset CIC-IDS2017 trên Hugging Face.

+ Dữ liệu được tạo từ các network flow trong một phiên làm việc có lưu lượng bình thường và tấn công DDoS.

2.2. Quy mô dữ liệu

+ Sau khi loại cột nhãn, notebook ghi nhận:

Thuộc tính	Giá trị
Tổng số mẫu	225.745
Số cột đầu vào ban đầu	78
Số lớp	2
Nhãn bình thường	BENIGN
Nhãn bất thường	ATTACK

+ Nhãn nhị phân được tạo bằng:

- Label == BENIGN → 0
- Label != BENIGN → 1

→ Do notebook chỉ dùng file DDoS buổi chiều thứ Sáu, lớp **ATTACK** về thực chất đại diện chủ yếu cho DDoS, không phải toàn bộ các kiểu xâm nhập trong CIC-IDS2017.

2.3. Nhóm thuộc tính

+ Dataset chứa các đặc trưng thống kê của network flow. Có thể chia thành các nhóm chính:

Nhóm	Ví dụ
Định danh flow	Flow ID, Source IP, Destination IP
Cổng và giao thức	Source Port, Destination Port, Protocol
Thời gian	Timestamp, Flow Duration
Thống kê packet	Total Forward Packets, Total Backward Packets
Kích thước packet	Packet Length Mean, Max, Min, Std
Tốc độ	Flow Bytes/s, Flow Packets/s
Cờ TCP	SYN, ACK, FIN, RST, PSH flag count
Inter-arrival time	Flow IAT, Fwd IAT, Bwd IAT
Header và payload	Header length, segment size
Trạng thái hoạt động	Active Mean, Idle Mean và các biến liên quan

+ Notebook loại các cột định danh nếu chúng xuất hiện:

- Flow ID
- Source IP
- Destination IP
- Src IP
- Dst IP

→ Việc loại IP và Flow ID giúp hạn chế tình trạng mô hình ghi nhớ trực tiếp nguồn hoặc đích của cuộc tấn công.

2.4. Phân bố nhãn

+ Phân bố toàn bộ dataset:

Nhãn	Tỷ lệ	Số mẫu xấp xỉ
BENIGN	43,2869%	97.718
ATTACK	56,7131%	128.027
Tổng	100%	225.745

2.5. Phân chia dữ liệu

Tập	Tổng mẫu	BENIGN	ATTACK	Tỷ lệ
Train	135.447	58.630	76.817	60%
Validation	45.149	19.544	25.605	20%
Test	45.149	19.544	25.605	20%
Tổng	225.745	97.718	128.027	100%

2.6. Tiền xử lý

+ Pipeline tiền xử lý gồm:

1. Chuẩn hóa tên cột bằng **strip()**.
2. Chuyển nhãn thành chữ hoa.
3. Chuyển các giá trị vô hạn thành NaN.
4. Loại cột định danh.

5. Tách thuộc tính số và phân loại.
6. Điền giá trị thiếu của thuộc tính số bằng median.
7. One-hot encode thuộc tính phân loại nếu có.
8. Chuẩn hóa bằng **StandardScaler**.

3. Pipeline

3.1. Quy trình tổng thể

CIC-IDS2017 CSV

|



Đọc dữ liệu bằng Pandas

|



Làm sạch tên cột và nhãn

|



Chuyển nhãn thành nhị phân

| — BENIGN → 0

| — DDoS → 1

|



Thay $\pm\infty$ bằng NaN

|



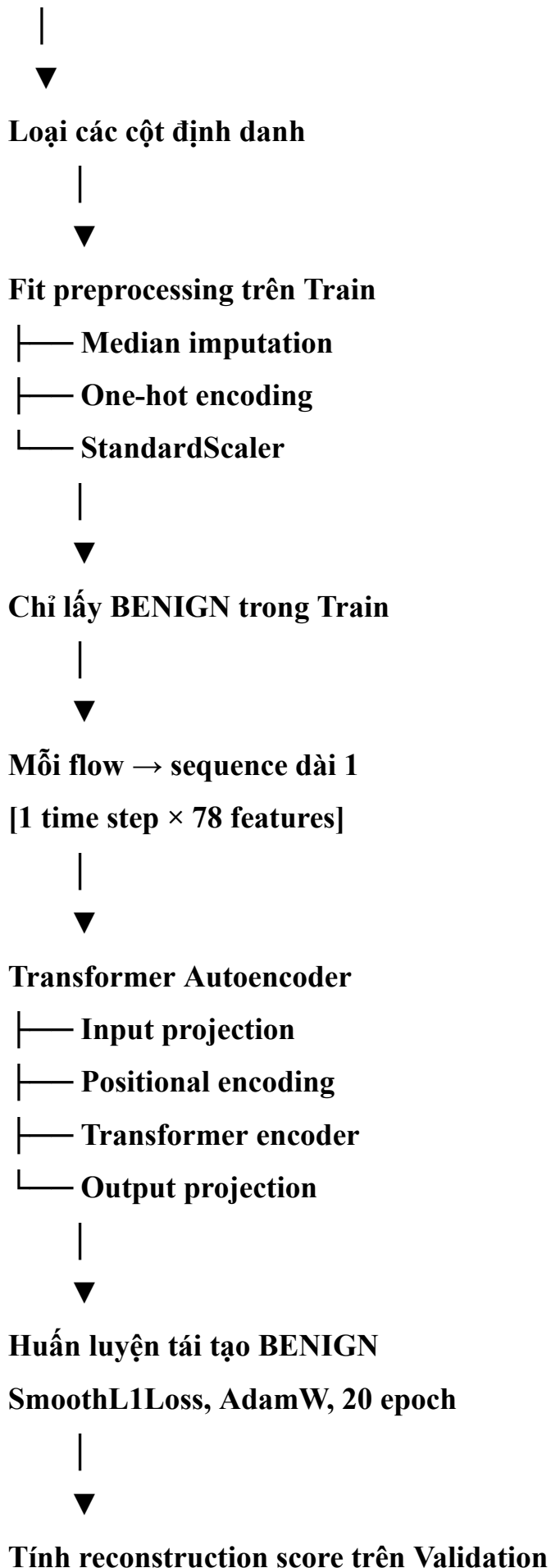
Chia có stratify

|



Train Val Test

60% 20% 20%



|
▼
Tối ưu threshold theo Validation F1

|
▼
Đánh giá một lần trên Test

|— **Classification report**

|— **Confusion matrix**

|— **F1-score**

|— **ROC-AUC**

|— **PR-AUC**

4. Model – Kiến trúc Transformer Autoencoder

4.1. Kiến trúc tổng thể

Input

(batch, sequence=1, features=78)

|
▼
Linear Projection

78 → 64

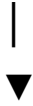
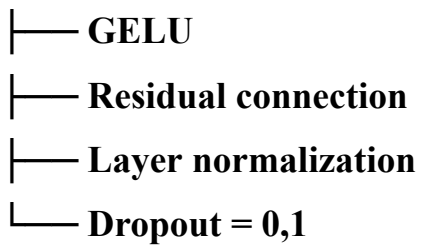
|
▼
Sinusoidal Positional Encoding

|
▼
Transformer Encoder × 3

|— **Multi-Head Self-Attention**

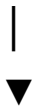
|— **8 attention heads**

|— **Feed-Forward: 64 → 256 → 64**



Linear Output Projection

64 → 78



Reconstructed input

(batch, sequence=1, features=78)

4.2. Cấu hình mô hình

Thành phần	Giá trị
Input dimension	78
dmodel	64
Attention heads	8
Số Transformer Encoder layer	3
Feed-forward dimension	256
Activation	GELU
Dropout	0,1
Output dimension	78
Loss	SmoothL1Loss
Optimizer	AdamW
Learning rate	0,001
Weight decay	10 ⁻⁵

Batch size	128
Epoch	20

5. So sánh kết quả

5.1. Train và validation loss

+ Train loss giảm: 0,055565 → 0,003794

+ Tương đương giảm khoảng: 93,17\%

+ Validation loss cũng giảm mạnh nhưng dao động từ epoch 9 trở đi. Validation loss thấp nhất xuất hiện tại epoch 15: **0,003578**

→ Notebook không lưu checkpoint tốt nhất nên kết quả cuối sử dụng mô hình epoch 20, dù validation loss tại epoch 20 cao hơn epoch 15.

5.2. Kết quả trên validation

Chỉ số	Giá trị
Threshold tối ưu	0,001724
Validation F1 tốt nhất	0,7945

5.3. Kết quả trên tập test

+ Confusion matrix:

Nhãn thật \ Nhãn dự đoán	BENIGN	BENIGN
BENIGN	12.190	7.354
ATTACK	3.979	21.626

TUẦN FINAL : PHÁT HIỆN BẤT THƯỜNG MẠNG TRÊN CSE-CIC-IDS2018

2. Dataset

2.1. Nguồn dữ liệu

+ Dự án sử dụng CSE-CIC-IDS2018, một dataset về phát hiện xâm nhập mạng.
Dữ liệu được lưu trong file nén:

- /content/drive/MyDrive/CICIDS2018.zip

2.2. Quá trình đọc dữ liệu

+ Do dataset gốc có dung lượng lớn, notebook đọc CSV theo từng chunk:

Tham số	Giá trị
Chunk size	100.000 dòng
Số dòng tối đa mỗi file	40.000
Số file CSV	10
Tổng số dòng đọc được	400.000
Tổng số cột ban đầu	85

+ Mỗi file đóng góp 40.000 dòng: $10 \times 40.000 = 400.000$

2.3. Phân bố dữ liệu ban đầu

+ Sau khi gộp 10 file:

Nhãn	Số mẫu	Tỷ lệ
BENIGN	215.946	53,99%
ATTACK	184.054	46,01%
Tổng	400.000	100%

2.4. Cân bằng dữ liệu

+ Notebook giới hạn mỗi lớp ở 50.000 mẫu bằng undersampling:

Nhãn	Trước cân bằng	Sau cân bằng
BENIGN	215.946	50.000
ATTACK	184.054	50.000
Tổng	400.000	100.000

→ Dataset cuối có tỷ lệ cân bằng hoàn toàn:

- $P(\text{BENIGN})=P(\text{ATTACK})=50\%$

→ Vì hai lớp cân bằng, class weight được tính bằng:

- ATTACK: 1,0
- BENIGN: 1,0

+ Undersampling giúp:

- Giảm thời gian huấn luyện.
- Tránh mô hình thiên lệch về một lớp.
- Giúp accuracy dễ diễn giải hơn.

→ Tuy nhiên, thao tác này loại bỏ 300.000 mẫu và có thể làm mất nhiều biến thiên quan trọng của dữ liệu thực tế.

2.5. Phân chia dữ liệu

+ Dataset 100.000 mẫu được chia có phân tầng:

Tập	Tổng mẫu	ATTACK	BENIGN	Tỷ lệ
Train	70.000	35.000	35.000	70%
Validation	15.000	7.500	7.500	15%
Test	15.000	7.500	7.500	15%
Tổng	100.000	50.000	50.000	100%

2.6. Tiền xử lý

+ Pipeline tiền xử lý gồm:

1. Chuẩn hóa tên cột.
2. Chuyển các giá trị về dạng số.
3. Giá trị không chuyển được thành số được đổi thành **NaN**.
4. Thay $\backslash(+\infty)$ và $\backslash(-\infty)$ bằng **NaN**.
5. Loại cột toàn giá trị thiếu hoặc không có biến thiên.
6. Điền giá trị thiếu bằng median của tập train.
7. Chuẩn hóa bằng **RobustScaler**.
8. Clip dữ liệu trong khoảng $(-10,10)$.

3. Pipeline

3.1. Quy trình tổng thể

CSE-CIC-IDS2018.zip

|



Giải nén 10 file CSV

|



Đọc từng file theo chunk

|— Tối đa 40.000 dòng/file

|— Chuẩn hóa tên cột

|— Xóa header bị lặp

└— Chuẩn hóa nhãn

|



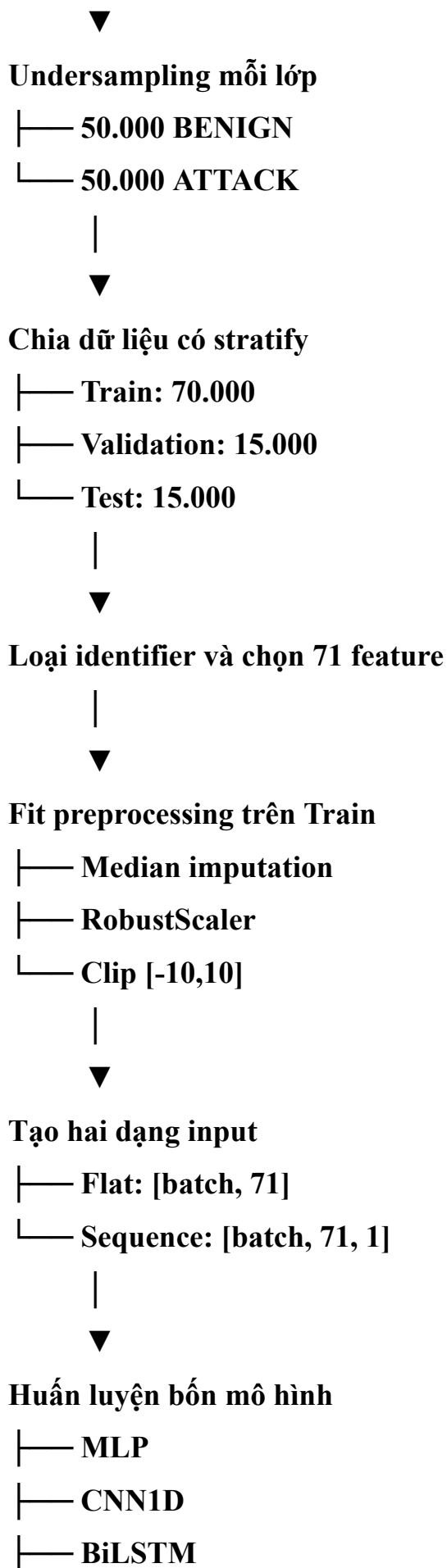
Gộp thành 400.000 flow

|



Gộp nhãn thành BENIGN/ATTACK

|



└─ Transformer

|



EarlyStopping + ReduceLROnPlateau

+ ModelCheckpoint

|



Đánh giá checkpoint tốt nhất trên Test

|

└─ Loss

└─ Accuracy

└─ Balanced accuracy

└─ Macro/Weighted F1

└─ ROC-AUC

└─ PR-AUC

|



Failure analysis và visualization

|

└─ Confusion matrix

└─ ROC/PR curve

└─ Attention

└─ Hidden representation

|



Lưu model và preprocessing artifact

3.2. Quy trình huấn luyện công bằng

+ Tất cả mô hình dùng chung:

Thành phần	Cấu hình
Epoch tối đa	15
Batch size	512
Learning rate ban đầu	0.001
Loss	Sparse Categorical Cross-Entropy
Optimizer	Adam
Early-stopping patience	4
Checkpoint criterion	Validation loss
Learning-rate reduction	Validation loss
Minimum learning rate	10^{-6}

→ **ModelCheckpoint** lưu epoch có validation loss tốt nhất. Sau huấn luyện, notebook tải checkpoint này và mới đánh giá trên **test**.

4. Model – Các kiến trúc đã thử nghiệm

4.1. MLP

+ MLP nhận trực tiếp vector 71 đặc trưng:

Input: 71

|



Dense 128

Batch Normalization

ReLU

Dropout 0,30

|

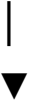


Dense 64

Batch Normalization

ReLU

Dropout 0,20



Dense 2 + Softmax

Thuộc tính	Thành phần
Kiểu input	Flat
Số tham số	18.178
Ưu điểm	Đơn giản, nhanh, phù hợp dữ liệu bảng
Hạn chế	Không mô hình hóa trực tiếp quan hệ có cấu trúc giữa feature

→ MLP là baseline tự nhiên nhất vì mỗi flow vốn là một vector thuộc tính cố định.

4.2. CNN1D

+ CNN1D xem 71 đặc trưng như một chuỗi:

Input: 71×1



Conv1D 64, kernel=5

BatchNorm + ReLU

MaxPooling1D



Conv1D 128, kernel=3

BatchNorm + ReLU

|



GlobalAveragePooling1D

Dropout 0,30

|



Dense 64 + ReLU

|



Dense 2 + Softmax

Thuộc tính	Giá trị
Kiểu input	Sequence
Số tham số	34.050
Ưu điểm	Học được mẫu cục bộ giữa các feature liền kề
Hạn chế	Kết quả phụ thuộc thứ tự cột

→ Vấn đề quan trọng là các feature mạng không tạo thành chuỗi thời gian. Hai feature đứng cạnh nhau trong CSV chưa chắc có quan hệ cục bộ có ý nghĩa.

4.3. BiLSTM

Input: 71×1

|



Bidirectional LSTM 48

return_sequences=True

|

Dropout 0,25



Bidirectional LSTM 24



Dense 64 + ReLU

Dropout 0,25



Dense 2 + Softmax

Thuộc tính	Giá trị
Kiểu input	Sequence
Số tham số	45.698
Ưu điểm	Tổng hợp quan hệ hai chiều trên dãy feature
Hạn chế	Huấn luyện chậm và thứ tự feature không phải thời gian

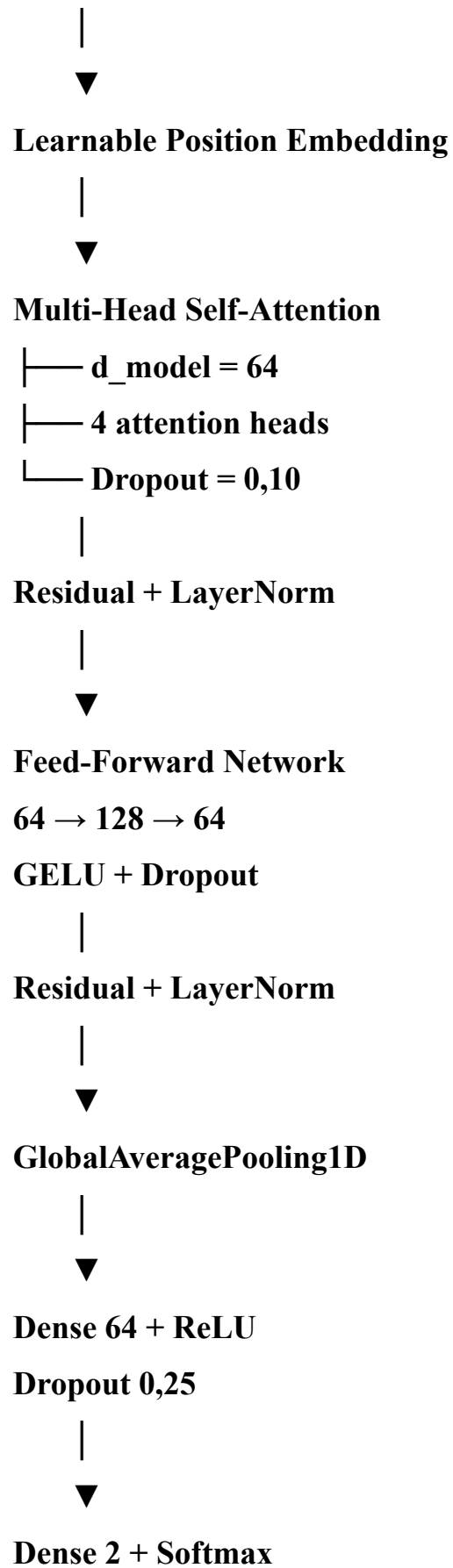
→ BiLSTM xem mỗi feature là một time step. Đây là cách khai thác tương tác giữa các thuộc tính, nhưng không nên diễn giải hidden state như trạng thái mạng thay đổi theo thời gian.

4.4. Transformer

Input: 71×1



Scalar Projection: $1 \rightarrow 64$



Thuộc tính	Giá trị
Sequence length	71 feature
Embedding dimension	64
Attention heads	4
Feed-forward dimension	128
Encoder block	1
Số tham số	42,434

5. So sánh kết quả

5.1. Kết quả tổng hợp trên test

Hạng	1	2	3	4
Model	MLP	BiLSTM	Transformer	CNN1D
Tham số	18.178	45.698	42.434	34.050
Epoch tốt nhất	14	15	15	13
Thời gian train	17,15 giây	65,26 giây	54,91 giây	23,85 giây
Test loss	0,1015	0,1102	0,1187	0,1316
Accuracy	96,79%	96,69%	96,49%	96,12%
Macro F1	96,79%	96,68%	96,49%	96,12%
ROC-AUC	98,59%	98,28%	98,25%	98,08%
PR-AUC	99,00%	98,82%	98,79%	98,66%

→ Do test set cân bằng, accuracy, balanced accuracy, macro F1 và weighted F1 của mỗi mô hình gần như bằng nhau.

5.2. Chênh lệch so với MLP

Model	Chênh lệch accuracy so với MLP	Thời gian so với MLP
MLP		1,00 lần
BiLSTM	-0,11 điểm %	3,81 lần
Transformer	-0,30 điểm %	3,20 lần
CNN1D	-0,67 điểm %	1,39 lần

+ MLP vừa:

- Có accuracy cao nhất.
- Có test loss thấp nhất.
- Có ROC-AUC và PR-AUC cao nhất.
- Có ít tham số nhất.
- Huấn luyện nhanh nhất.

→ Kết quả cho thấy kiến trúc phức tạp hơn không mặc nhiên hiệu quả hơn.

5.3. Kết quả chi tiết của MLP

Lớp	Precision	Recall	F1-score	Support
ATTACK	99,66%	93,91%	96,70%	7.500
BENIGN	94,24%	99,68%	96,88%	7.500
Macro average	96,95%	96,79%	96,79%	15.000

+ Confusion matrix:

Nhãn thật \ Nhãn dự đoán	ATTACK	BENIGN

ATTACK	7.043	457
BENIGN	24	7.476

5.4. Tại sao MLP tốt nhất?

+ Kết quả này hợp lý vì dữ liệu là tabular data:

- Mỗi dòng là một flow độc lập.
- Mỗi cột có ý nghĩa riêng.
- Thứ tự cột không phải thứ tự thời gian.
- Các feature đã là những thống kê tổng hợp có chất lượng cao.
- MLP có thể học tương tác phi tuyến giữa toàn bộ feature mà không áp đặt cấu trúc chuỗi.

+ CNN1D chỉ học pattern cục bộ giữa các cột đứng gần nhau. BiLSTM và Transformer cũng phải học trên một thứ tự feature do con người quy định. Cấu trúc này không nhất thiết phản ánh quan hệ tự nhiên của traffic.

→ Do đó, inductive bias đơn giản của MLP phù hợp hơn với dữ liệu hiện tại.

6. Failure Analysis

6.1. Phân bố lỗi của mô hình tốt nhất

+ MLP dự đoán sai: $481 / 15,000 = 3,21 \%$

+ Các lỗi gồm:

- ATTACK → BENIGN  457 lỗi (95,01%)
- BENIGN → ATTACK  24 lỗi (4,99%)

Loại lỗi	Số lượng	Tỷ lệ trong tổng lỗi
ATTACK bị dự đoán	457	95,01%

thành BENIGN		
BENIGN bị dự đoán thành ATTACK	24	4,99%
Tổng	481	100%

6.2. False negative

+ Trong 7.500 mẫu tấn công, mô hình bỏ sót 457 mẫu:

- $FNR = 6,09\%$ hay $Recall_{Attack} = 93,91\%$

→ Điều này có nghĩa cứ 100 flow tấn công, hệ thống bỏ sót khoảng sáu flow.

→ Đối với hệ thống an ninh mạng, đây là rủi ro quan trọng hơn accuracy tổng thể. Một tấn công không bị phát hiện có thể gây thiệt hại lớn hơn một cảnh báo giả.

6.3. False positive

+ Trong 7.500 mẫu bình thường, 24 mẫu bị báo nhầm:

- $FPR = 0,32\%$

→ Đây là tỷ lệ cảnh báo giả rất thấp.

+ Mô hình hiện có xu hướng thận trọng khi gán nhãn **ATTACK**:

- Khi đã dự đoán **ATTACK**, precision đạt 99,66%.
- Tuy nhiên, mô hình bỏ qua một phần attack để duy trì precision cao.

6.4. Trade-off precision–recall

+ Đối với lớp ATTACK:

- Precision: 99,66%.

- Recall: 93,91%.

+ Chênh lệch : 5,75%

→ Điều này cho thấy threshold mặc định ưu tiên giảm false alarm hơn là phát hiện toàn bộ attack.

+ Nếu ứng dụng ưu tiên an toàn, có thể giảm threshold dự đoán ATTACK nhằm:

- Tăng attack recall.
- Giảm false negative.
- Chấp nhận tăng một phần false positive.

KẾT LUẬN

Dự án final đã xây dựng thành công một pipeline phát hiện bất thường mạng hoàn chỉnh trên CSE-CIC-IDS2018. Hệ thống đọc dữ liệu lớn theo chunk, cân bằng lớp, chia train-validation-test, tiền xử lý tránh rò rỉ và so sánh bốn kiến trúc trong cùng điều kiện.

+ Kết quả tốt nhất thuộc về MLP:

- Test accuracy: 96,79%.
- Macro F1: 96,79%.
- ROC-AUC: 98,59%.
- PR-AUC: 99,00%.
- Test loss: 0,1015.
- Thời gian huấn luyện: 17,15 giây.
- Số tham số: 18.178.

+ MLP vượt BiLSTM, Transformer và CNN1D dù có ít tham số và thời gian huấn luyện thấp nhất. Điều này cho thấy dữ liệu network flow hiện tại phù hợp với mô hình tabular hơn các kiến trúc áp đặt cấu trúc chuỗi lên thứ tự feature.

+ Mô hình tạo rất ít false positive—24/7.500 mẫu BENIGN—nhưng bỏ sót 457/7.500 mẫu ATTACK. Vì vậy, hướng cải tiến quan trọng nhất là tối ưu threshold để tăng attack recall.

+ Báo cáo cũng cần ghi nhận ba giới hạn chính:

1. Dữ liệu được chia ngẫu nhiên theo flow thay vì theo ngày.
2. Test set đã được cân bằng nên chưa phản ánh base rate thực tế.
3. Mô hình tốt nhất được chọn dựa trên test Macro-F1 thay vì validation

→ Dù còn những giới hạn này, notebook final đã hoàn thiện tốt hơn rõ rệt các notebook theo tuần: có quy trình tái sử dụng, bốn kiến trúc đối chứng, checkpoint, metric toàn diện, failure analysis, attention visualization, live demo và artifact phục vụ triển khai.